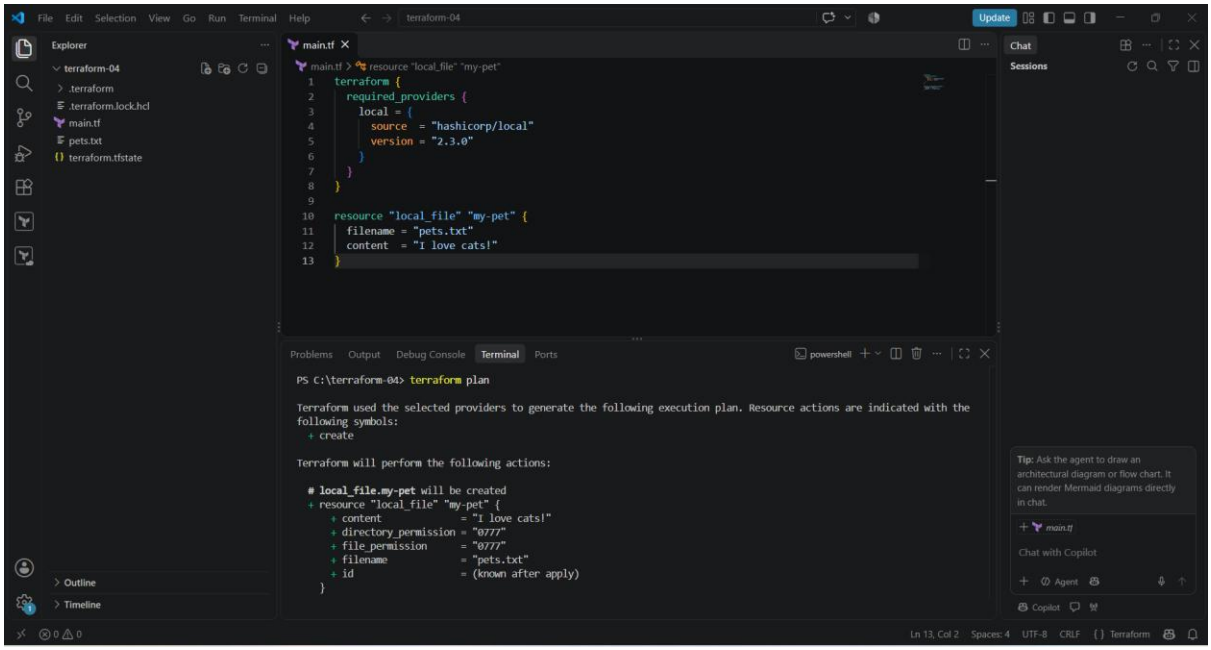
1. Watch **Terraform-04** video.

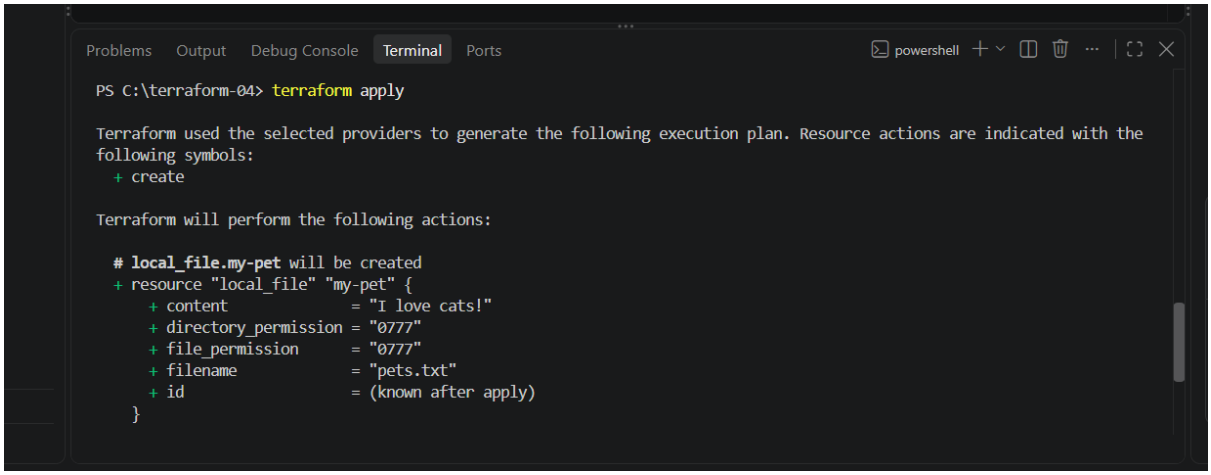
   Completed.

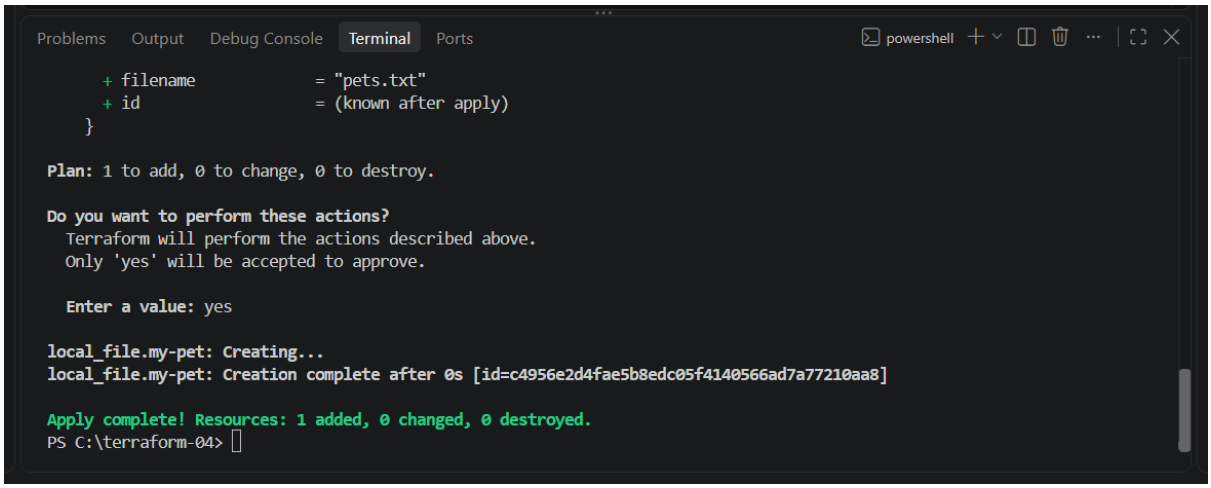
2. **Execute the script shown in the video.**

1.Create a File Using the Terraform Local Provider

1. Created a working directory named terraform-04 in VS Code.
2. Created the main.tf configuration file.
3. Configured the hashicorp/local provider with version 2.3.0.
4. Created a local_file resource named my-pet.
5. Used the filename attribute to specify pets.txt and the content attribute to write I love cats!.
6. Initialized the Terraform working directory using terraform init.
7. Validated the configuration using terraform validate.
8. Reviewed the execution plan using terraform plan.
9. Executed terraform apply to create the file.
10. Verified that pets.txt was created with the specified content.



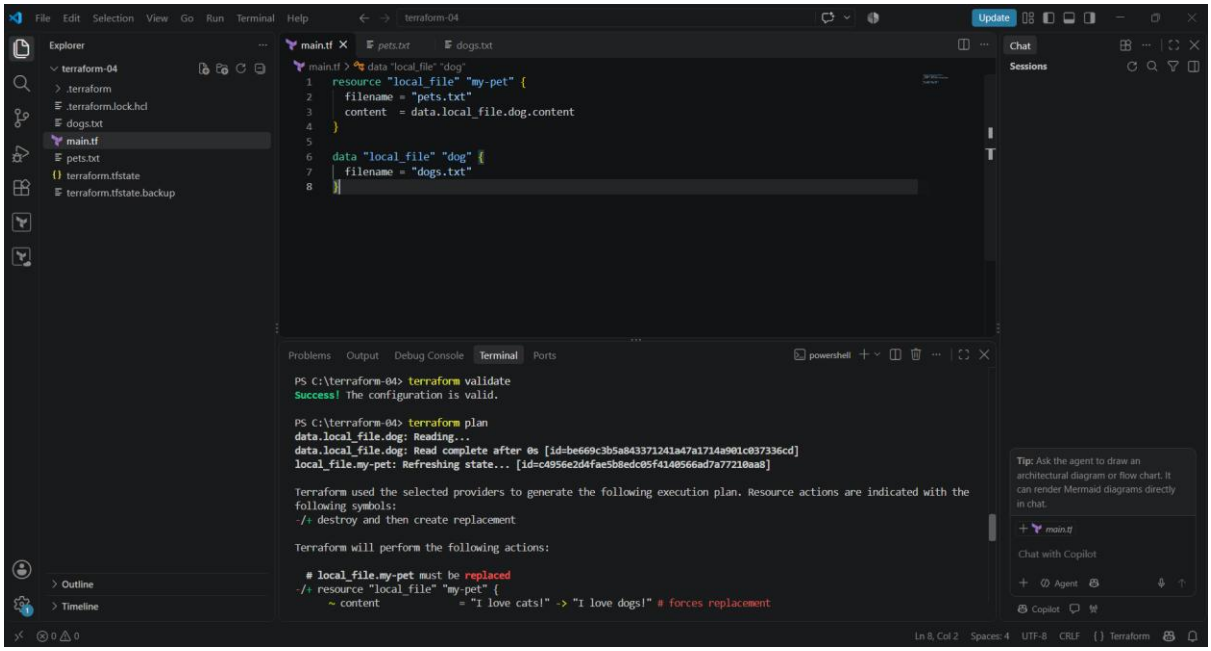




Data:

Use a Terraform Data Source

1. Created the dogs.txt file manually and added the content I love dogs!.
2. Updated main.tf to define the local_file data source named dog.
3. Used the filename attribute to specify the existing dogs.txt file.
4. Accessed the file content using data.local_file.dog.content.
5. Configured the local_file resource to write the retrieved content into pets.txt.
6. Executed terraform plan to review the changes.
7. Executed terraform apply to update the destination file.
8. Verified that pets.txt contained I love dogs!.




```
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you
run "terraform apply" now.
PS C:\terraform-04> terraform apply
data.local_file.dog: Reading...
data.local_file.dog: Read complete after 0s [id=be669c3b5a843371241a47a1714a901c037336cd]
local_file.my-pet: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # local_file.my-pet must be replaced
-/+ resource "local_file" "my-pet" {
      ~ content              = "I love cats!" -> "I love dogs!" # forces replacement
      ~ id                   = "c4956e2d4fae5b8edc05f4140566ad7a77210aa8" -> (known after apply)
```

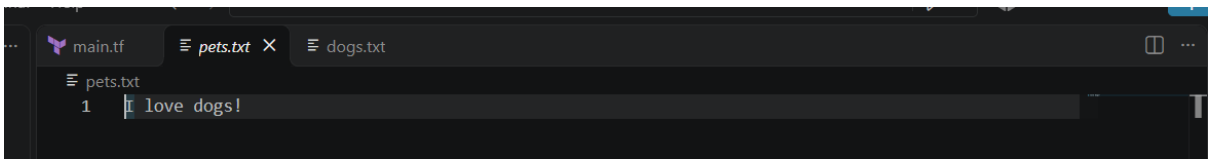
```
  # local_file.my-pet must be replaced
-/+ resource "local_file" "my-pet" {
      ~ content              = "I love cats!" -> "I love dogs!" # forces replacement
      ~ id                   = "c4956e2d4fae5b8edc05f4140566ad7a77210aa8" -> (known after apply)
        # (3 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

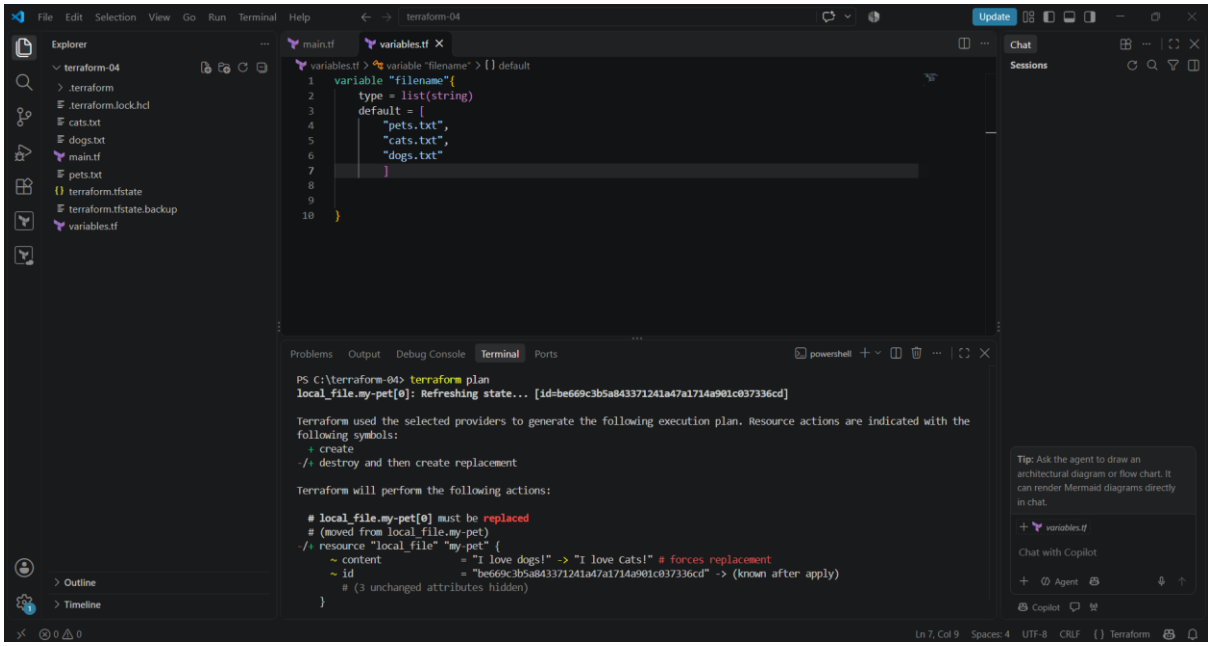
local_file.my-pet: Destroying... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
```

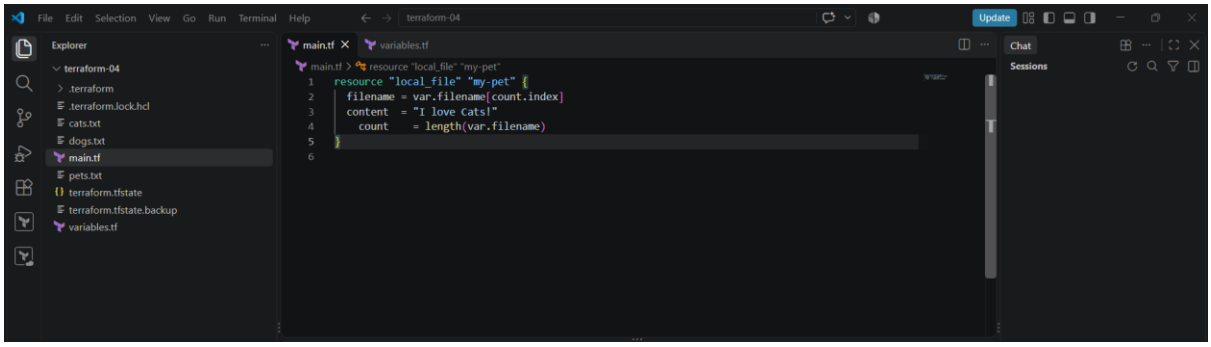


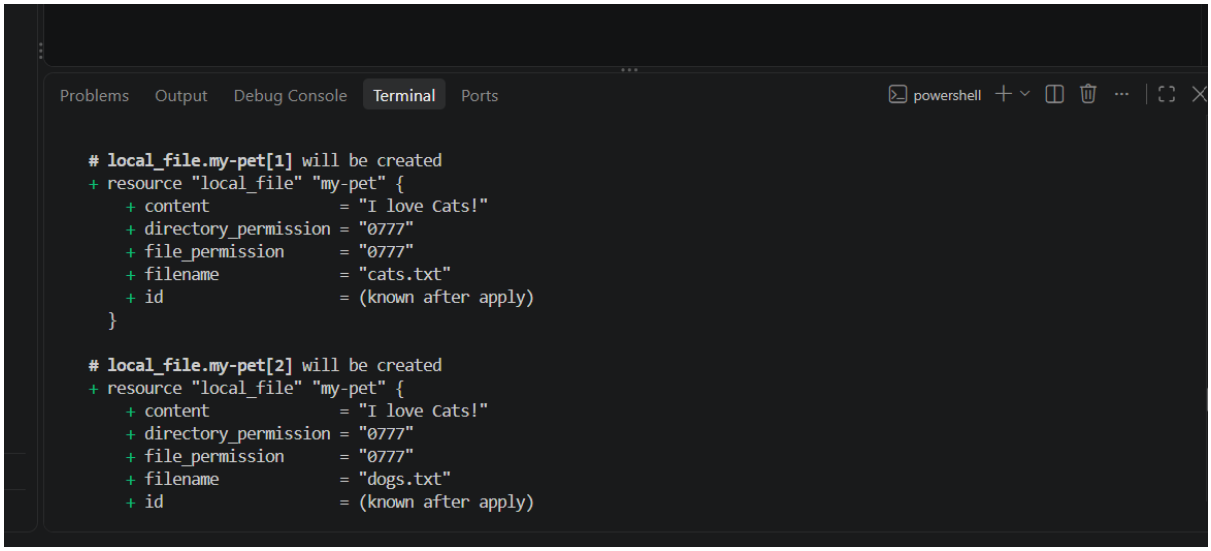
Count:

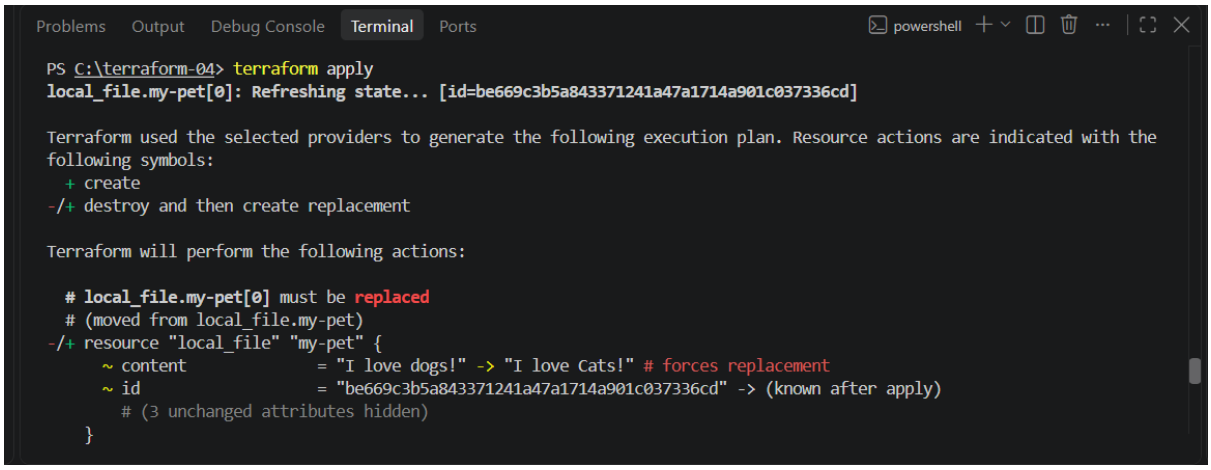
Create Multiple Files Using the count Meta-Argument

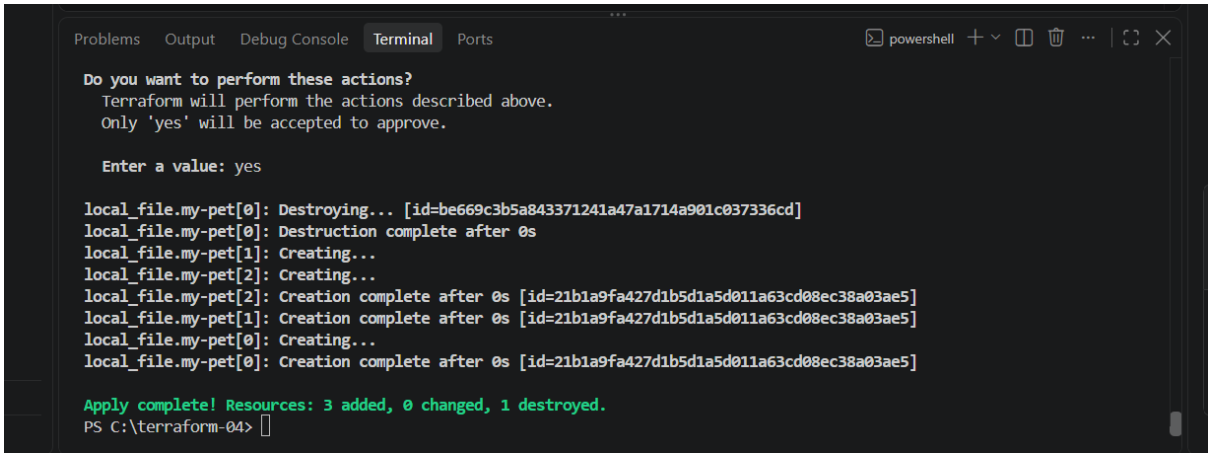
1. Created a variables.tf file to define the filename variable.
2. Assigned the list ["pets.txt", "cats.txt", "dogs.txt"] as the variable's default value.
3. Updated main.tf to create a local_file resource.
4. Used the count meta-argument with count = length(var.filename) to create one resource instance for each filename.
5. Used filename = var.filename[count.index] to assign a different filename to each resource instance.
6. Configured the content attribute with I love cats!.
7. Executed terraform plan to review the resources Terraform would create.
8. Executed terraform apply to create the files.
9. Verified that pets.txt, cats.txt, and dogs.txt were created.




File Edit Selection View Go Run Terminal Help
terraform-04
Update
Explorer
terraform-04
.terraform
.terraform.lock.hcl
cats.txt
dogs.txt
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.backup
variables.tf
main.tf
variables.tf
resource "local_file" "my-pet"
1 resource "local_file" "my-pet" {
2 filename = var.filename[count.index]
3 content = "I love Cats!"
4 count = length(var.filename)
5 }
6
Chat
Sessions


Problems
Output
Debug Console
Terminal
Ports
powershell
local_file.my-pet[1] will be created
+ resource "local_file" "my-pet" {
+ content = "I love Cats!"
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "cats.txt"
+ id = (known after apply)
}
local_file.my-pet[2] will be created
+ resource "local_file" "my-pet" {
+ content = "I love Cats!"
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "dogs.txt"
+ id = (known after apply)

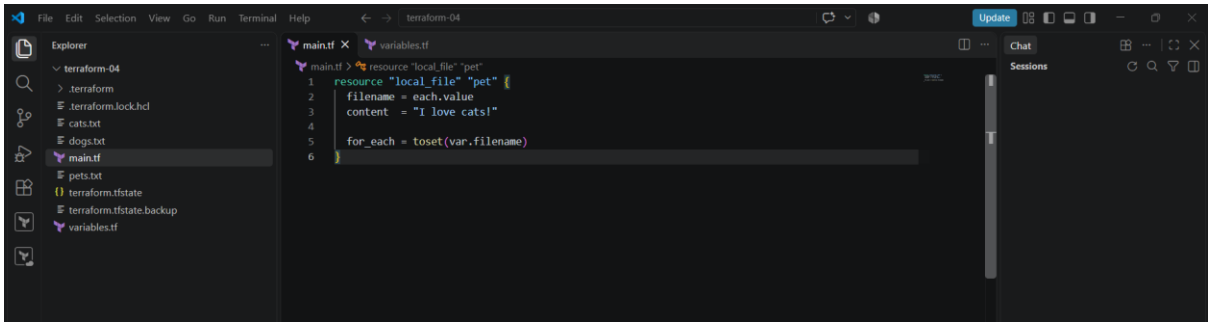

Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\terraform-04> terraform apply
local_file.my-pet[0]: Refreshing state... [id=be669c3b5a843371241a47a1714a901c037336cd]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
-/+ destroy and then create replacement
Terraform will perform the following actions:
local_file.my-pet[0] must be replaced
(moved from local_file.my-pet)
-/+ resource "local_file" "my-pet" {
~ content = "I love dogs!" -> "I love Cats!" # forces replacement
~ id = "be669c3b5a843371241a47a1714a901c037336cd" -> (known after apply)
(3 unchanged attributes hidden)
}

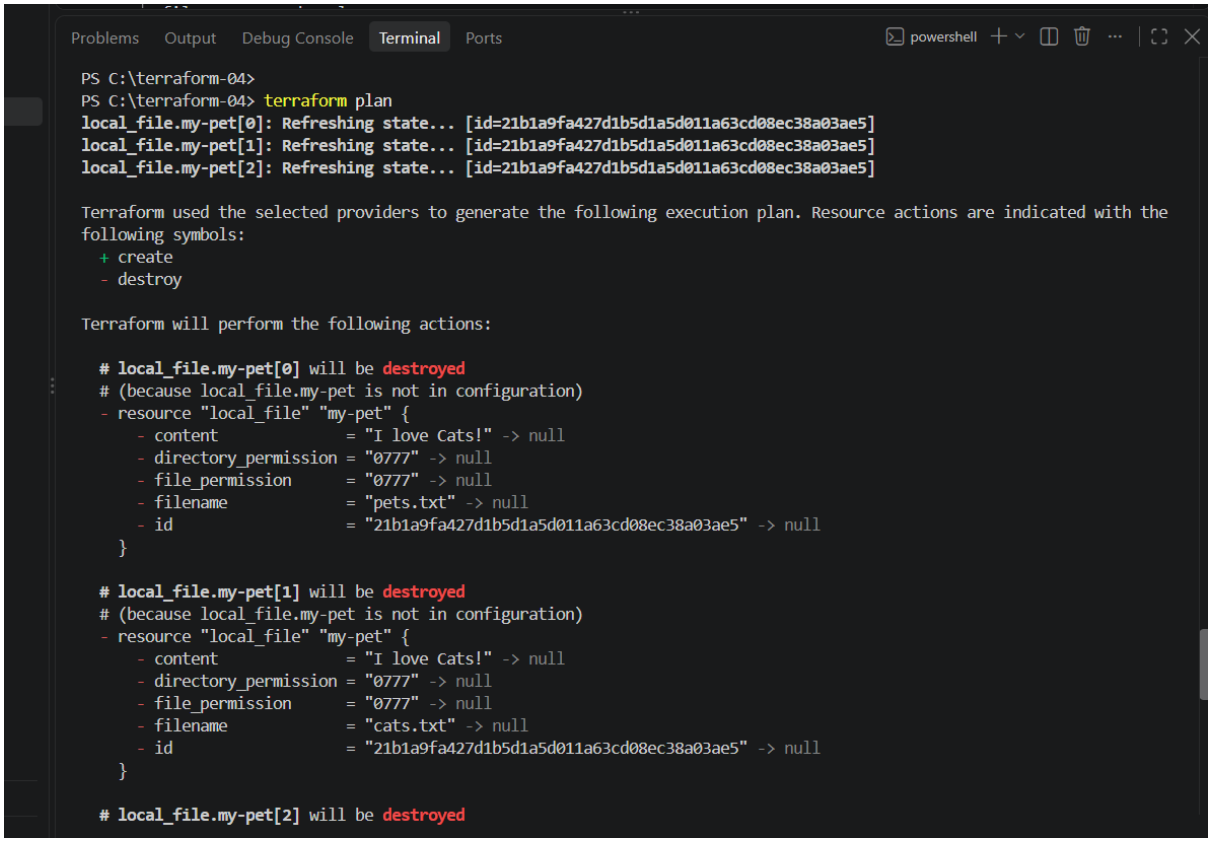


For_each:

Create Multiple Files Using the for_each Meta-Argument

1. Reused the filename list defined in variables.tf.
2. Updated the local_file resource configuration in main.tf.
3. Used for_each = toset(var.filename) to create a resource instance for each unique filename.
4. Used filename = each.value to assign the filename to each instance.
5. Configured the content attribute with I love cats!.
6. Executed terraform plan to review the planned resource changes.
7. Executed terraform apply to apply the configuration.
8. Verified the resources using their individual keys, such as local_file.pet["cats.txt"], local_file.pet["dogs.txt"], and local_file.pet["pets.txt"].





```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

PS C:\terraform-04>
PS C:\terraform-04> terraform plan
local_file.my-pet[0]: Refreshing state... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
local_file.my-pet[1]: Refreshing state... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
local_file.my-pet[2]: Refreshing state... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]

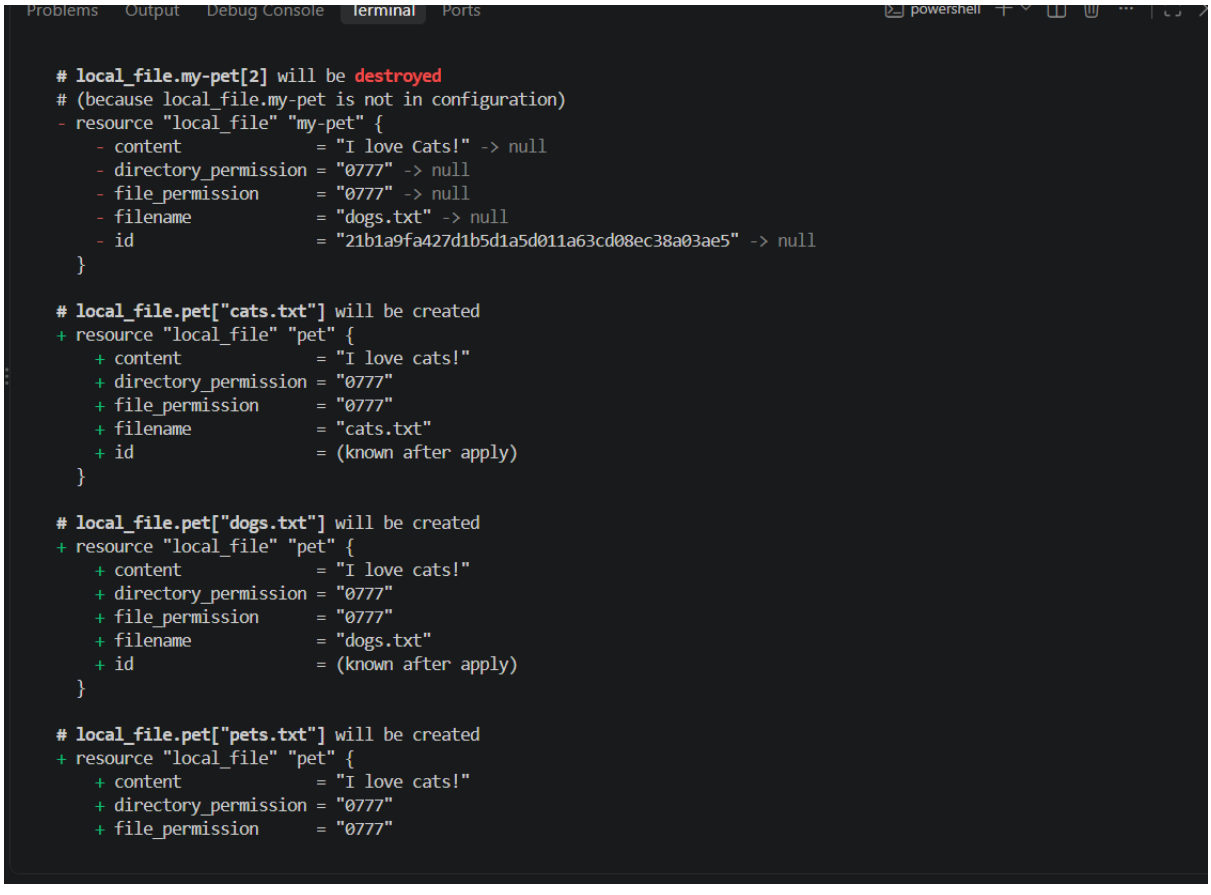
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create
  - destroy

Terraform will perform the following actions:

  # local_file.my-pet[0] will be destroyed
  # (because local_file.my-pet is not in configuration)
  - resource "local_file" "my-pet" {
      - content              = "I love Cats!" -> null
      - directory_permission = "0777" -> null
      - file_permission      = "0777" -> null
      - filename             = "pets.txt" -> null
      - id                   = "21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5" -> null
    }

  # local_file.my-pet[1] will be destroyed
  # (because local_file.my-pet is not in configuration)
  - resource "local_file" "my-pet" {
      - content              = "I love Cats!" -> null
      - directory_permission = "0777" -> null
      - file_permission      = "0777" -> null
      - filename             = "cats.txt" -> null
      - id                   = "21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5" -> null
    }

  # local_file.my-pet[2] will be destroyed
```



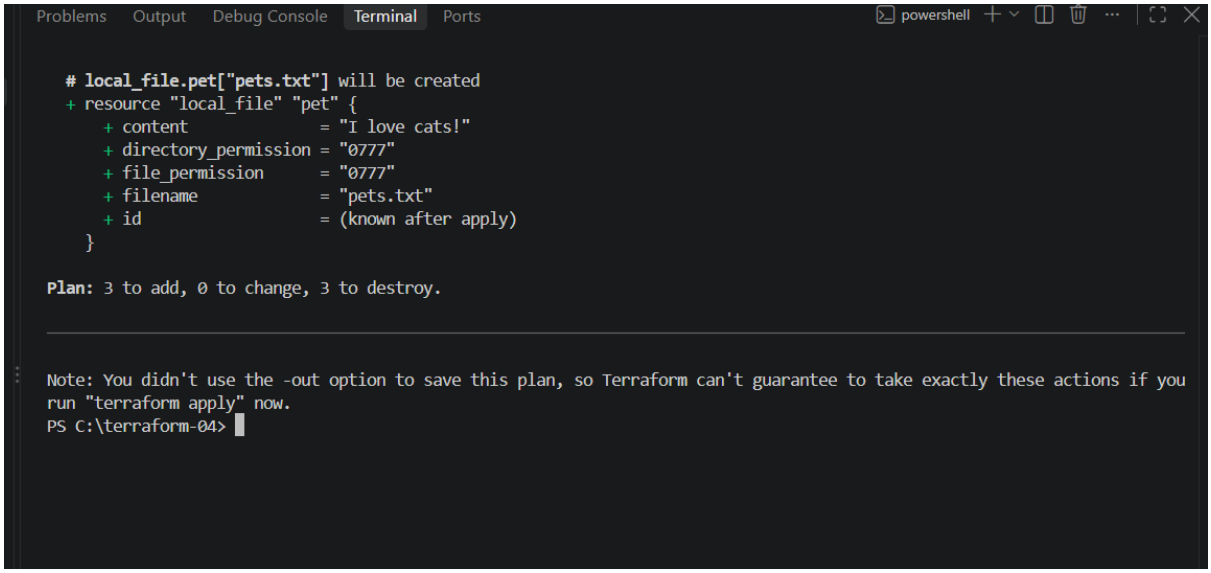
```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

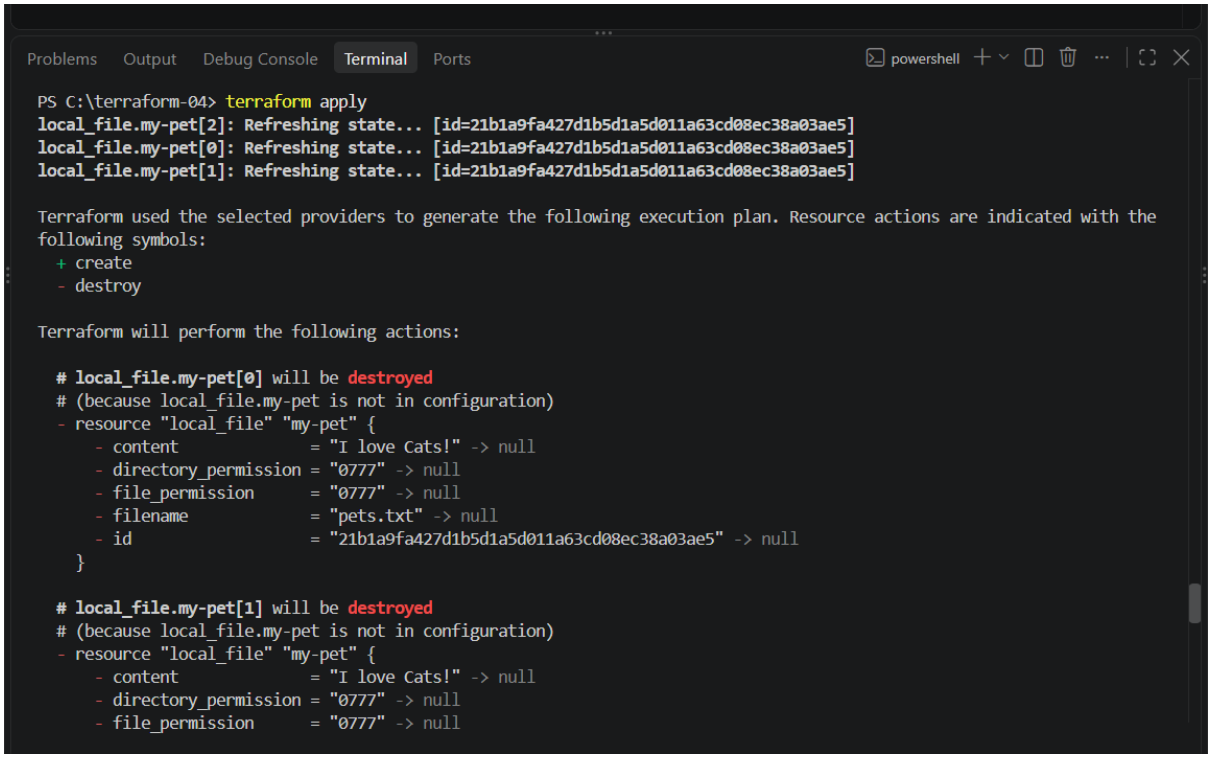
  # local_file.my-pet[2] will be destroyed
  # (because local_file.my-pet is not in configuration)
  - resource "local_file" "my-pet" {
      - content              = "I love Cats!" -> null
      - directory_permission = "0777" -> null
      - file_permission      = "0777" -> null
      - filename             = "dogs.txt" -> null
      - id                   = "21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5" -> null
    }

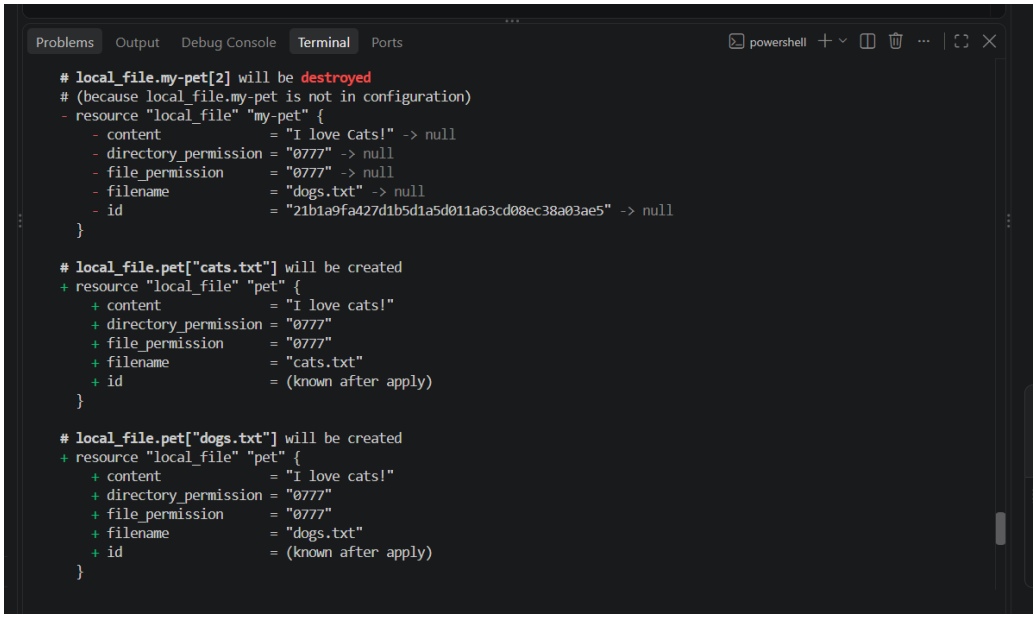
  # local_file.pet["cats.txt"] will be created
  + resource "local_file" "pet" {
      + content              = "I love cats!"
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "cats.txt"
      + id                   = (known after apply)
    }

  # local_file.pet["dogs.txt"] will be created
  + resource "local_file" "pet" {
      + content              = "I love cats!"
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "dogs.txt"
      + id                   = (known after apply)
    }

  # local_file.pet["pets.txt"] will be created
  + resource "local_file" "pet" {
      + content              = "I love cats!"
      + directory_permission = "0777"
      + file_permission      = "0777"
```


Problems Output Debug Console Terminal Ports
powershell
local_file.pet["pets.txt"] will be created
+ resource "local_file" "pet" {
+ content = "I love cats!"
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "pets.txt"
+ id = (known after apply)
}
Plan: 3 to add, 0 to change, 3 to destroy.
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\terraform-04>

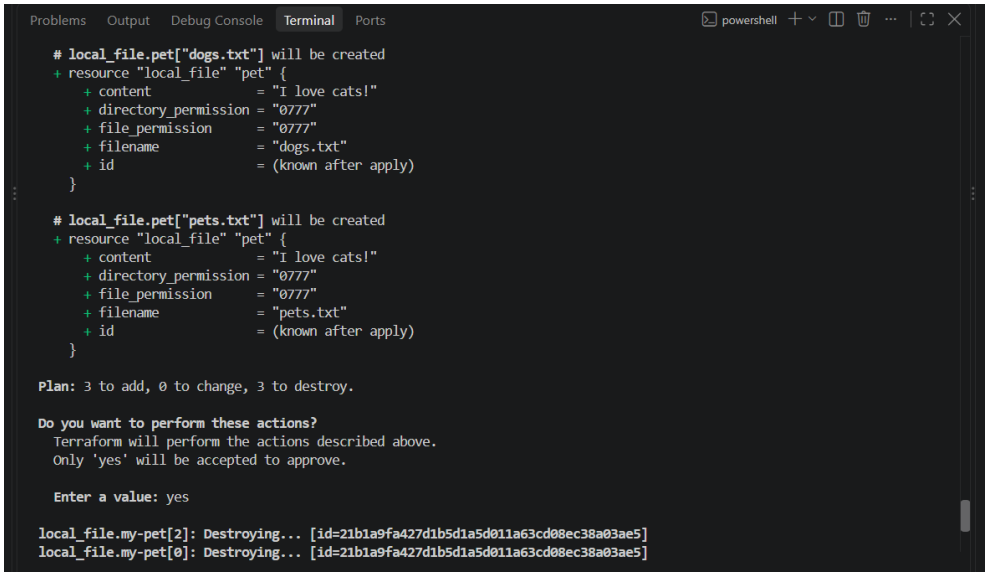

Problems Output Debug Console Terminal Ports
powershell
PS C:\terraform-04> terraform apply
local_file.my-pet[2]: Refreshing state... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
local_file.my-pet[0]: Refreshing state... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
local_file.my-pet[1]: Refreshing state... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy
Terraform will perform the following actions:
local_file.my-pet[0] will be destroyed
(because local_file.my-pet is not in configuration)
- resource "local_file" "my-pet" {
- content = "I love Cats!" -> null
- directory_permission = "0777" -> null
- file_permission = "0777" -> null
- filename = "pets.txt" -> null
- id = "21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5" -> null
}
local_file.my-pet[1] will be destroyed
(because local_file.my-pet is not in configuration)
- resource "local_file" "my-pet" {
- content = "I love Cats!" -> null
- directory_permission = "0777" -> null
- file_permission = "0777" -> null




```
  # local_file.my-pet[2] will be destroyed
  # (because local_file.my-pet is not in configuration)
  - resource "local_file" "my-pet" {
      - content              = "I love Cats!" -> null
      - directory_permission = "0777" -> null
      - file_permission      = "0777" -> null
      - filename             = "dogs.txt" -> null
      - id                   = "21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5" -> null
    }

  # local_file.pet["cats.txt"] will be created
  + resource "local_file" "pet" {
      + content              = "I love cats!"
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "cats.txt"
      + id                   = (known after apply)
    }

  # local_file.pet["dogs.txt"] will be created
  + resource "local_file" "pet" {
      + content              = "I love cats!"
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "dogs.txt"
      + id                   = (known after apply)
    }
```




```
  # local_file.pet["dogs.txt"] will be created
  + resource "local_file" "pet" {
      + content              = "I love cats!"
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "dogs.txt"
      + id                   = (known after apply)
    }

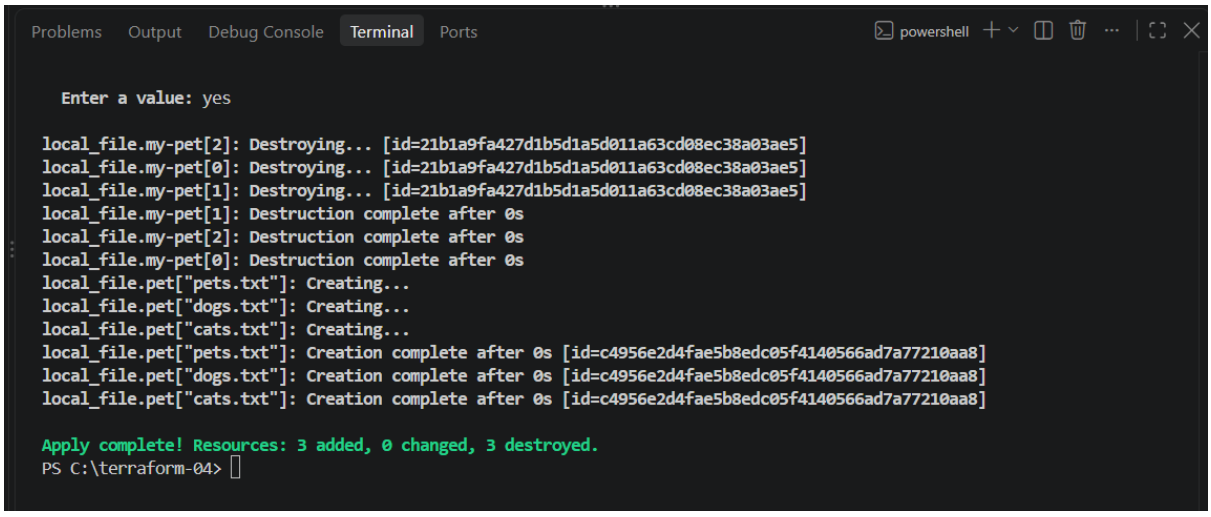
  # local_file.pet["pets.txt"] will be created
  + resource "local_file" "pet" {
      + content              = "I love cats!"
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "pets.txt"
      + id                   = (known after apply)
    }

Plan: 3 to add, 0 to change, 3 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my-pet[2]: Destroying... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
local_file.my-pet[0]: Destroying... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
```




```
  Enter a value: yes

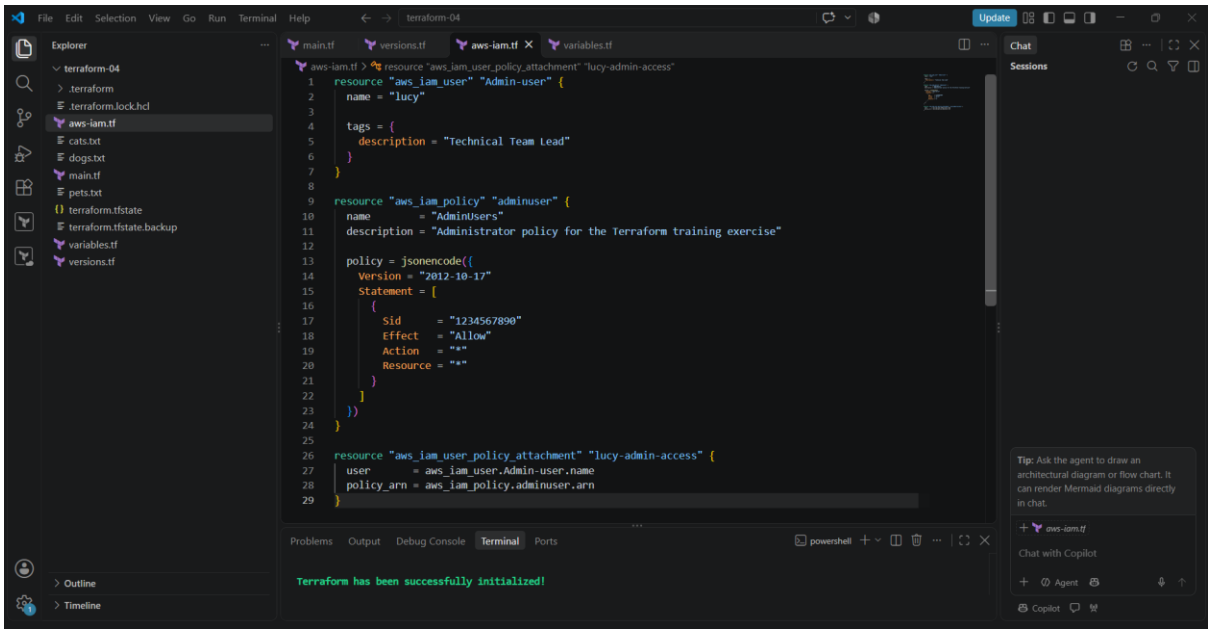
local_file.my-pet[2]: Destroying... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
local_file.my-pet[0]: Destroying... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
local_file.my-pet[1]: Destroying... [id=21b1a9fa427d1b5d1a5d011a63cd08ec38a03ae5]
local_file.my-pet[1]: Destruction complete after 0s
local_file.my-pet[2]: Destruction complete after 0s
local_file.my-pet[0]: Destruction complete after 0s
local_file.pet["pets.txt"]: Creating...
local_file.pet["dogs.txt"]: Creating...
local_file.pet["cats.txt"]: Creating...
local_file.pet["pets.txt"]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.pet["dogs.txt"]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.pet["cats.txt"]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]

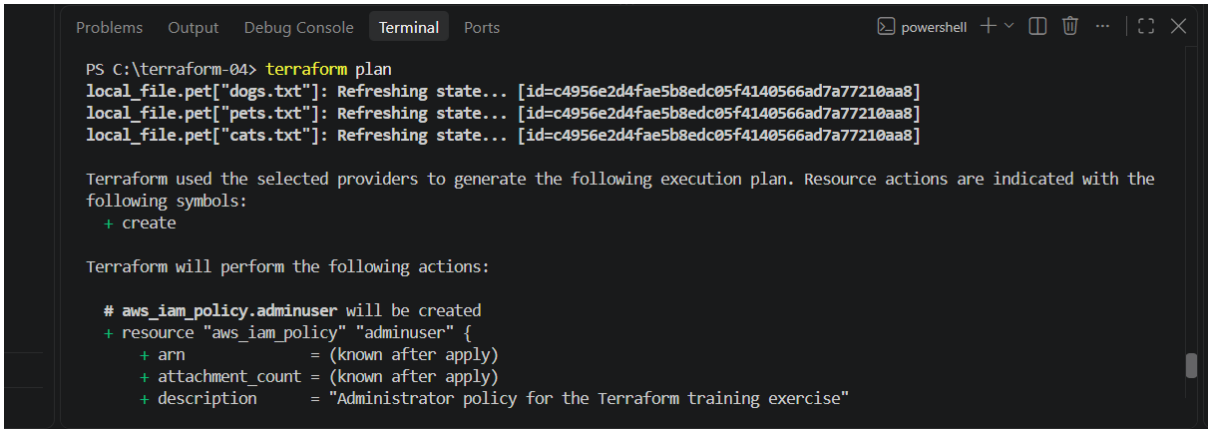
Apply complete! Resources: 3 added, 0 changed, 3 destroyed.
PS C:\terraform-04>
```

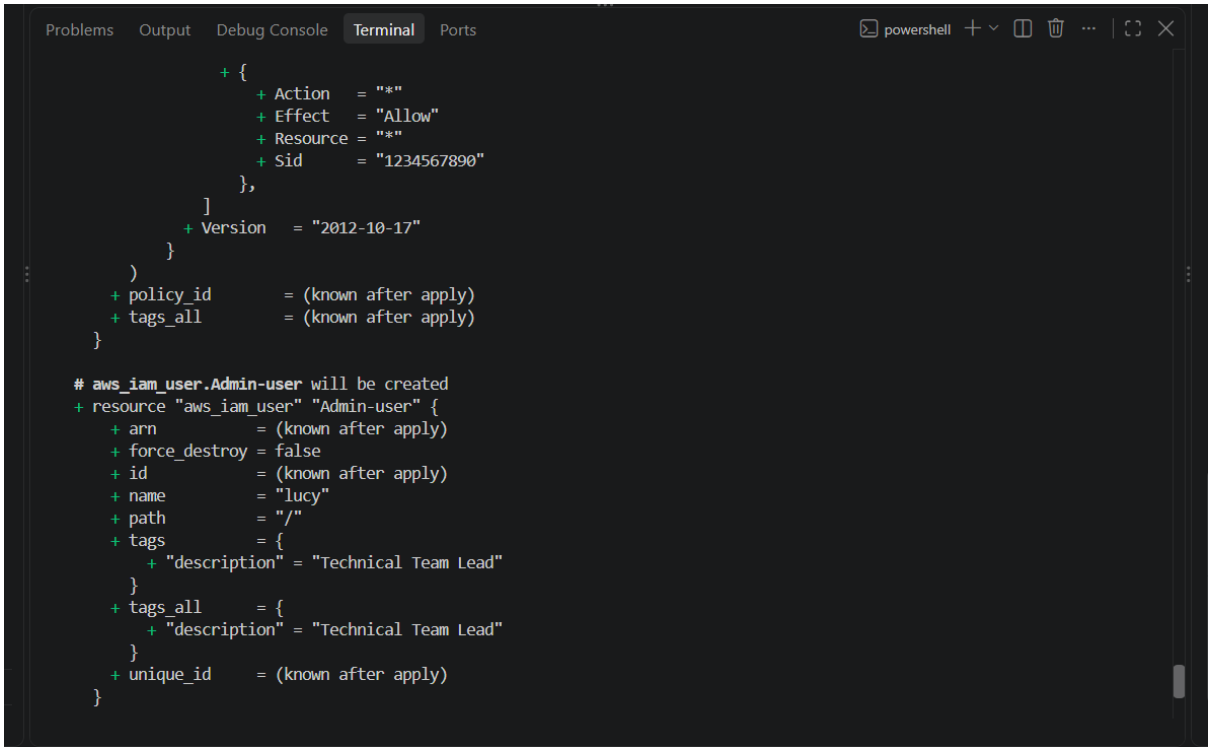
IAM User creation.

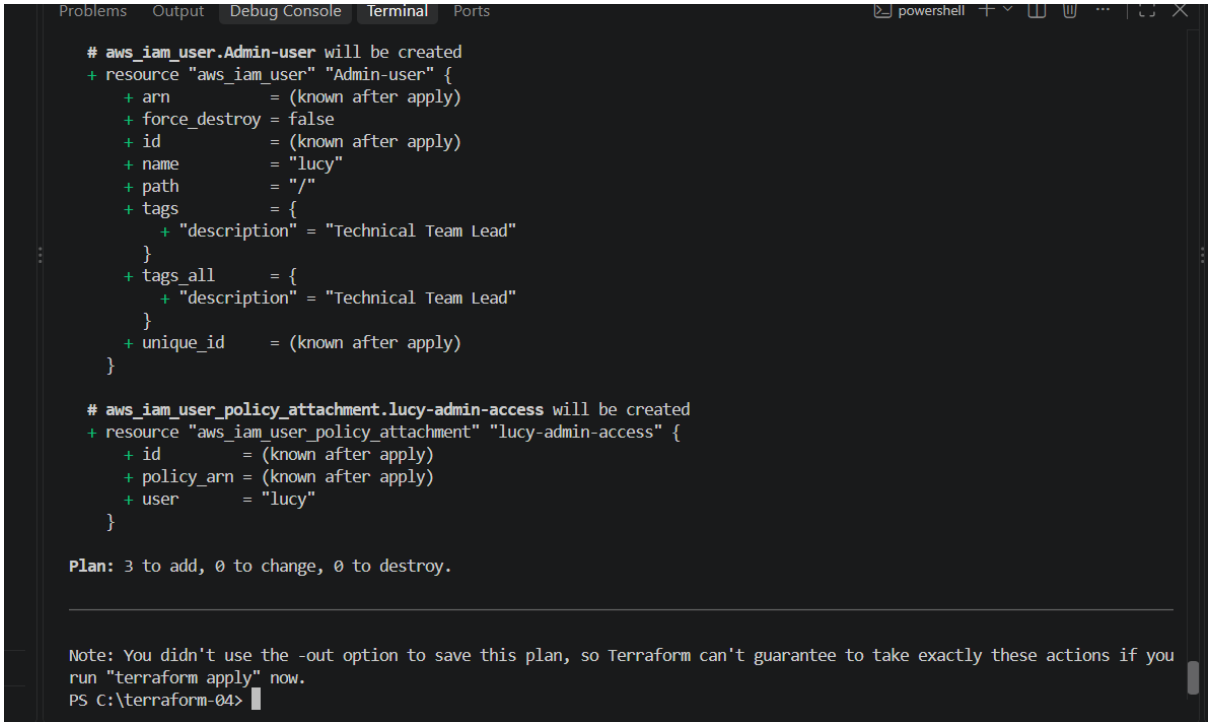
Create an AWS IAM User and Policy Using Terraform

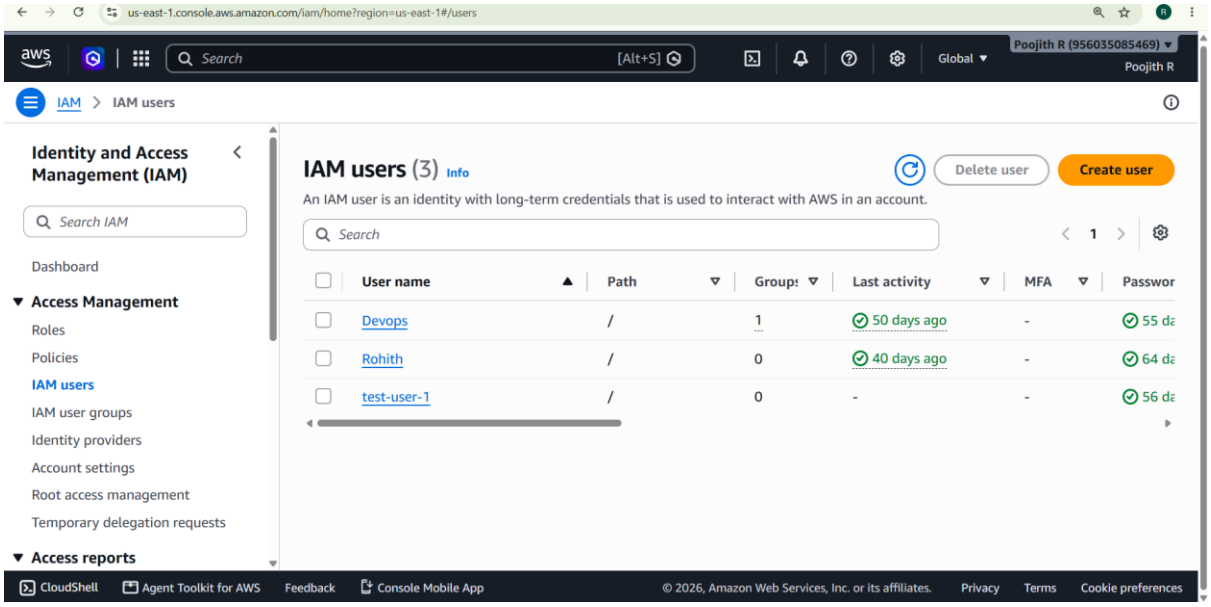
1. Configured the AWS provider in the Terraform configuration.
2. Defined an aws_iam_user resource to create an IAM user named lucy.
3. Defined an aws_iam_policy resource named AdminUsers.
4. Configured the policy document with "Action": "*" and "Resource": "*".
5. Initialized Terraform and validated the configuration.
6. Reviewed the planned changes using terraform plan.
7. Executed terraform apply to create the IAM resources.
8. Opened the AWS Management Console and navigated to IAM.
9. Verified that the user lucy and policy AdminUsers were visible.

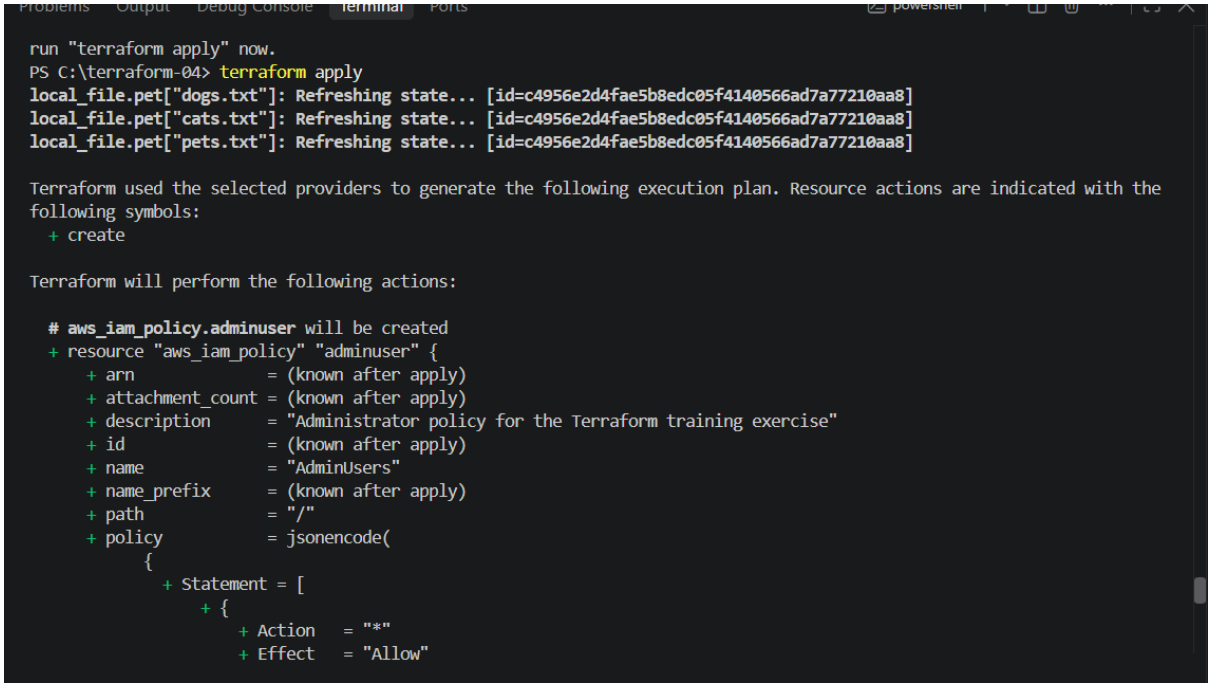



Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\terraform-04> terraform plan
local_file.pet["dogs.txt"]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.pet["pets.txt"]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.pet["cats.txt"]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
aws_iam_policy.adminuser will be created
+ resource "aws_iam_policy" "adminuser" {
+ arn = (known after apply)
+ attachment_count = (known after apply)
+ description = "Administrator policy for the Terraform training exercise"


Problems
Output
Debug Console
Terminal
Ports
powershell
+ {
+ Action = "*"
+ Effect = "Allow"
+ Resource = "*"
+ Sid = "1234567890"
},
]
+ Version = "2012-10-17"
}
)
+ policy_id = (known after apply)
+ tags_all = (known after apply)
}
aws_iam_user.Admin-user will be created
+ resource "aws_iam_user" "Admin-user" {
+ arn = (known after apply)
+ force_destroy = false
+ id = (known after apply)
+ name = "lucy"
+ path = "/"
+ tags = {
+ "description" = "Technical Team Lead"
}
+ tags_all = {
+ "description" = "Technical Team Lead"
}
+ unique_id = (known after apply)
}


Problems
Output
Debug Console
Terminal
Ports
powershell
aws_iam_user.Admin-user will be created
+ resource "aws_iam_user" "Admin-user" {
+ arn = (known after apply)
+ force_destroy = false
+ id = (known after apply)
+ name = "lucy"
+ path = "/"
+ tags = {
+ "description" = "Technical Team Lead"
}
+ tags_all = {
+ "description" = "Technical Team Lead"
}
+ unique_id = (known after apply)
}
aws_iam_user_policy_attachment.lucy-admin-access will be created
+ resource "aws_iam_user_policy_attachment" "lucy-admin-access" {
+ id = (known after apply)
+ policy_arn = (known after apply)
+ user = "lucy"
}
Plan: 3 to add, 0 to change, 0 to destroy.
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\terraform-04>


us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/users
aws
Search
[Alt+S]
Global
Poojith R (956035085469)
Poojith R
IAM
IAM users
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
IAM users (3) Info
An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.
Delete user
Create user
Search
User name
Path
Groups
Last activity
MFA
Passwor
Devops / 1 50 days ago - 55 da
Rohith / 0 40 days ago - 64 da
test-user-1 / 0 - - 56 da
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

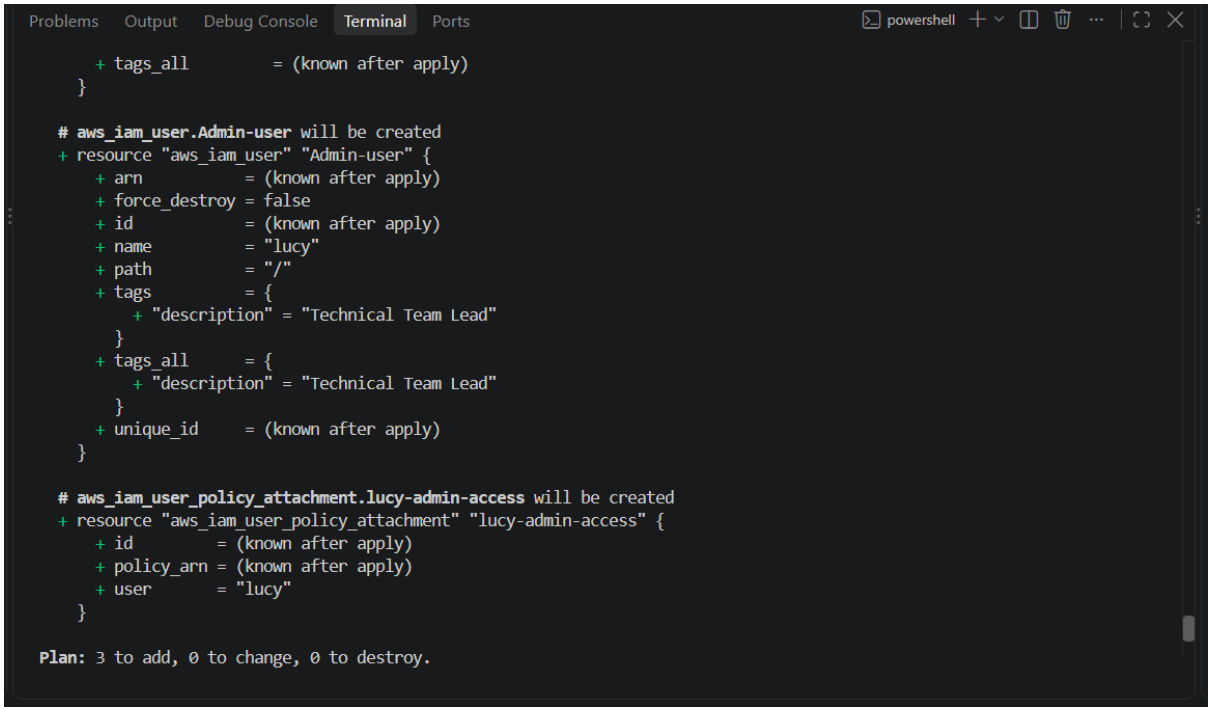


```
run "terraform apply" now.
PS C:\terraform-04> terraform apply
local_file.pet["dogs.txt"]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.pet["cats.txt"]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.pet["pets.txt"]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_policy.adminuser will be created
  + resource "aws_iam_policy" "adminuser" {
      + arn              = (known after apply)
      + attachment_count = (known after apply)
      + description      = "Administrator policy for the Terraform training exercise"
      + id               = (known after apply)
      + name             = "AdminUsers"
      + name_prefix      = (known after apply)
      + path             = "/"
      + policy           = jsonencode(
            {
              + Statement = [
                  + {
                      + Action   = "*"
                      + Effect   = "Allow"
```

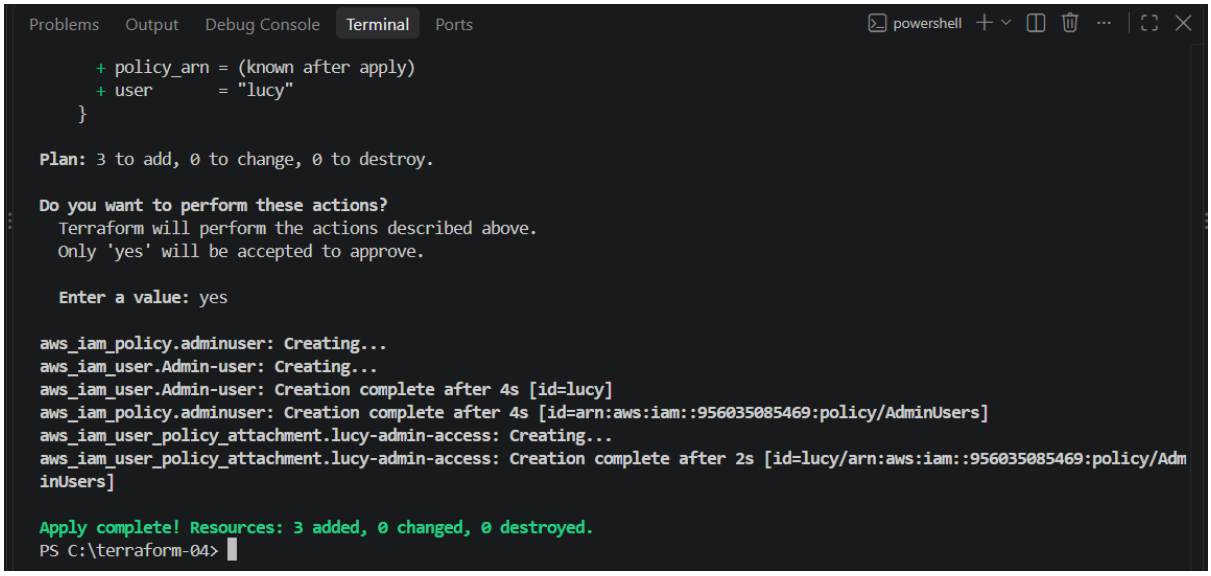


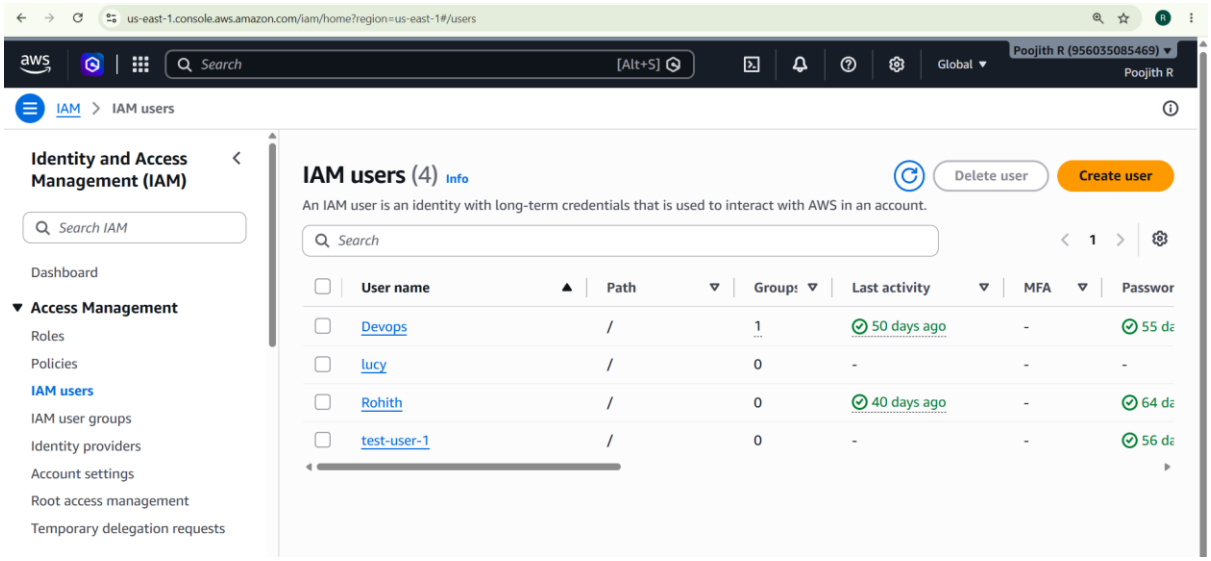
```
      + tags_all         = (known after apply)
    }

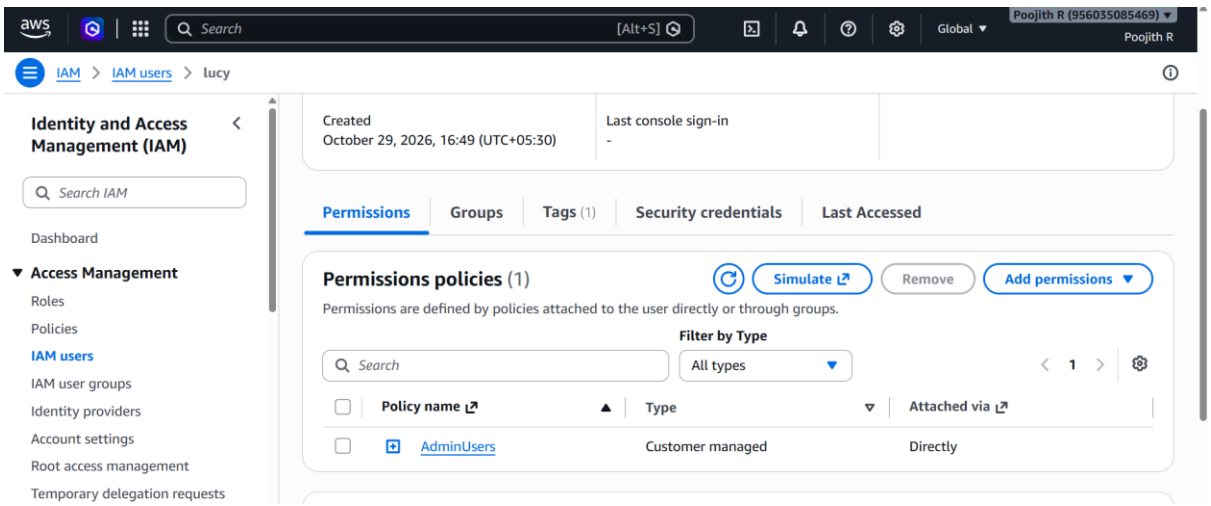
  # aws_iam_user.Admin-user will be created
  + resource "aws_iam_user" "Admin-user" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "lucy"
      + path          = "/"
      + tags          = {
          + "description" = "Technical Team Lead"
        }
      + tags_all      = {
          + "description" = "Technical Team Lead"
        }
      + unique_id     = (known after apply)
    }

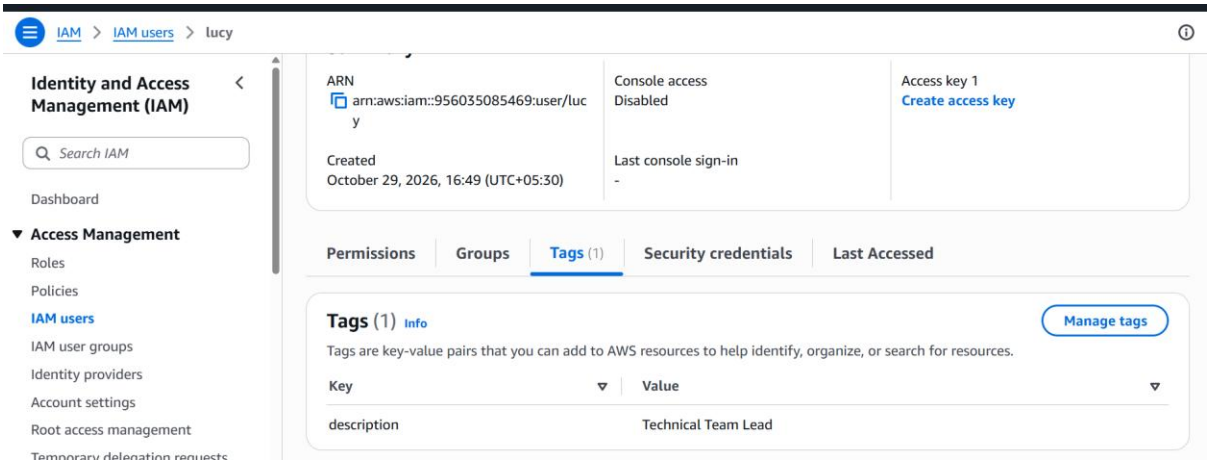
  # aws_iam_user_policy_attachment.lucy-admin-access will be created
  + resource "aws_iam_user_policy_attachment" "lucy-admin-access" {
      + id         = (known after apply)
      + policy_arn = (known after apply)
      + user       = "lucy"
    }

Plan: 3 to add, 0 to change, 0 to destroy.
```


Problems
Output
Debug Console
Terminal
Ports
powershell
+ policy_arn = (known after apply)
+ user = "lucy"
}
Plan: 3 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_iam_policy.adminuser: Creating...
aws_iam_user.Admin-user: Creating...
aws_iam_user.Admin-user: Creation complete after 4s [id=lucy]
aws_iam_policy.adminuser: Creation complete after 4s [id=arn:aws:iam::956035085469:policy/AdminUsers]
aws_iam_user_policy_attachment.lucy-admin-access: Creating...
aws_iam_user_policy_attachment.lucy-admin-access: Creation complete after 2s [id=lucy/arn:aws:iam::956035085469:policy/AdminUsers]
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
PS C:\terraform-04>


us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/users
Search
[Alt+S]
Global
Poojith R (956035085469)
Poojith R
IAM > IAM users
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
IAM users (4) Info
Delete user
Create user
An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.
User name
Path
Groups
Last activity
MFA
Passwor
Devops / 1 50 days ago - 55 da
lucy / 0 - - -
Rohith / 0 40 days ago - 64 da
test-user-1 / 0 - - 56 da


Search
[Alt+S]
Global
Poojith R (956035085469)
Poojith R
IAM > IAM users > lucy
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Created
October 29, 2026, 16:49 (UTC+05:30)
Last console sign-in
-
Permissions
Groups
Tags (1)
Security credentials
Last Accessed
Permissions policies (1)
Simulate
Remove
Add permissions
Permissions are defined by policies attached to the user directly or through groups.
Filter by Type
Search
All types
Policy name
Type
Attached via
AdminUsers
Customer managed
Directly



3. **Integrate Terraform in Jenkins using the Terraform plugin.**

Implementation Steps

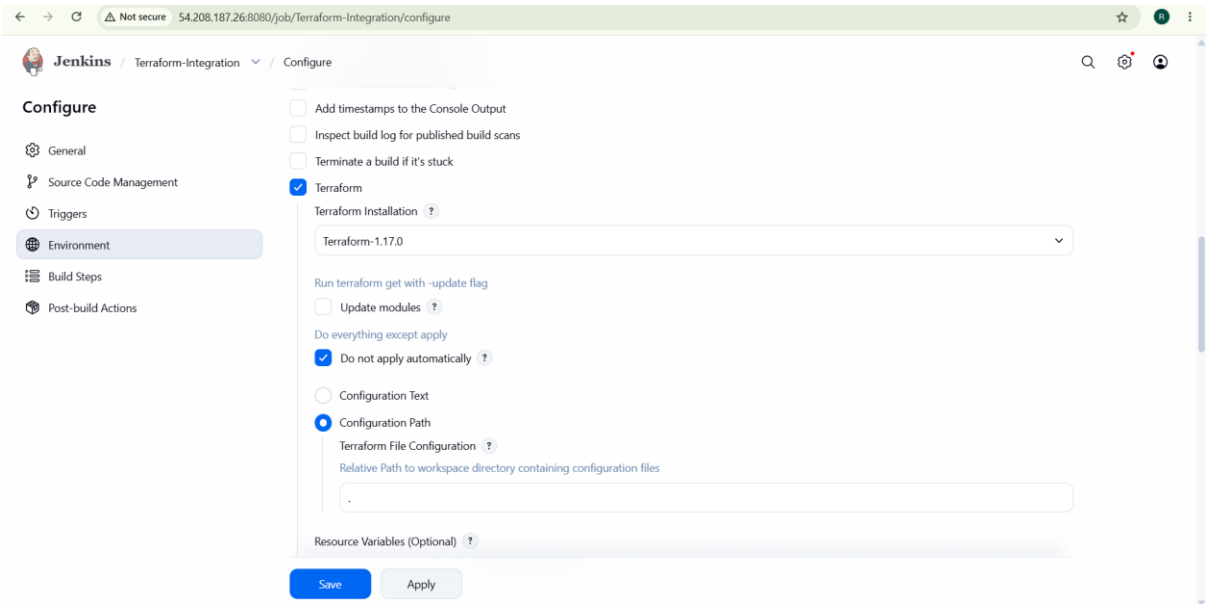
1. Accessed the Jenkins dashboard and opened **Manage Jenkins → Plugins**.
2. Installed the Terraform plugin required for the integration.
3. Opened **Manage Jenkins → Tools** and configured the Terraform installation with the name Terraform-1.17.0.
4. Used /usr/bin as the Terraform installation directory and left automatic installation unchecked.
5. Verified that Terraform was installed on the Jenkins EC2 instance and was accessible at /usr/bin/terraform.
6. Created a Freestyle Jenkins job named Terraform-Integration.
7. Selected the configured Terraform installation, Terraform-1.17.0, in the Terraform build environment.
8. Configured the Terraform plugin's configuration path as . to refer to the job workspace.
9. Enabled **Do not apply automatically** to prevent the plugin from automatically applying the Terraform configuration.
10. Added an Execute shell build step that created a main.tf file in the Jenkins workspace using a shell heredoc.
11. Configured the Terraform file to use the Local provider and a local_file resource named my_pet, with pets.txt as the filename and i love dogs as the content.

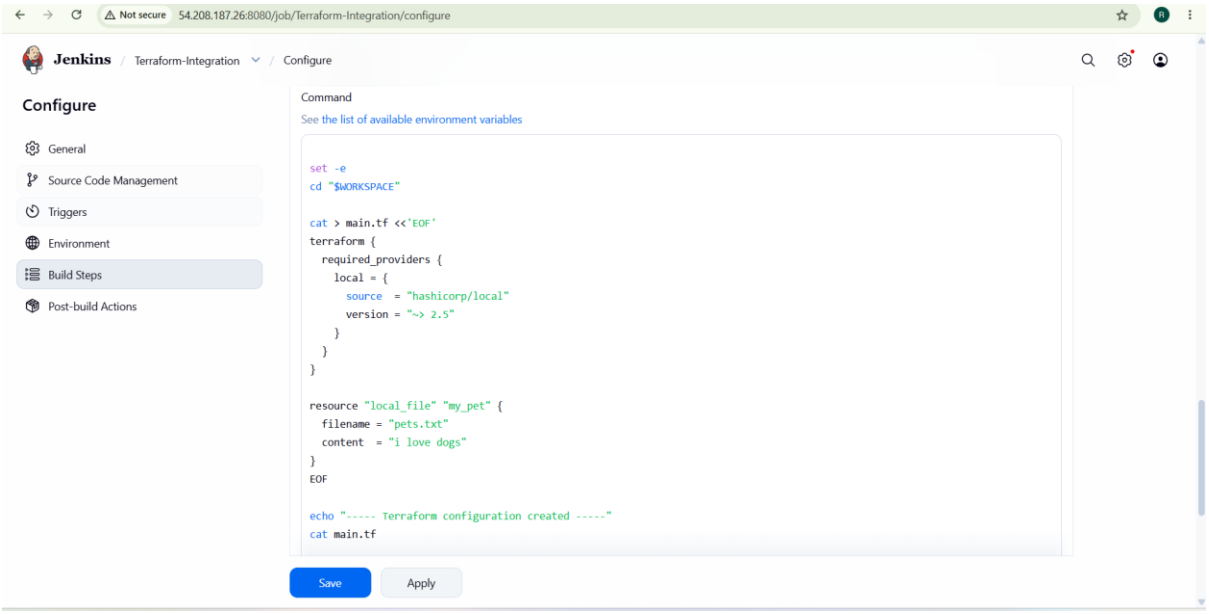
12. Printed the generated Terraform configuration in the Jenkins build log for verification.
13. Saved the job and executed the build.

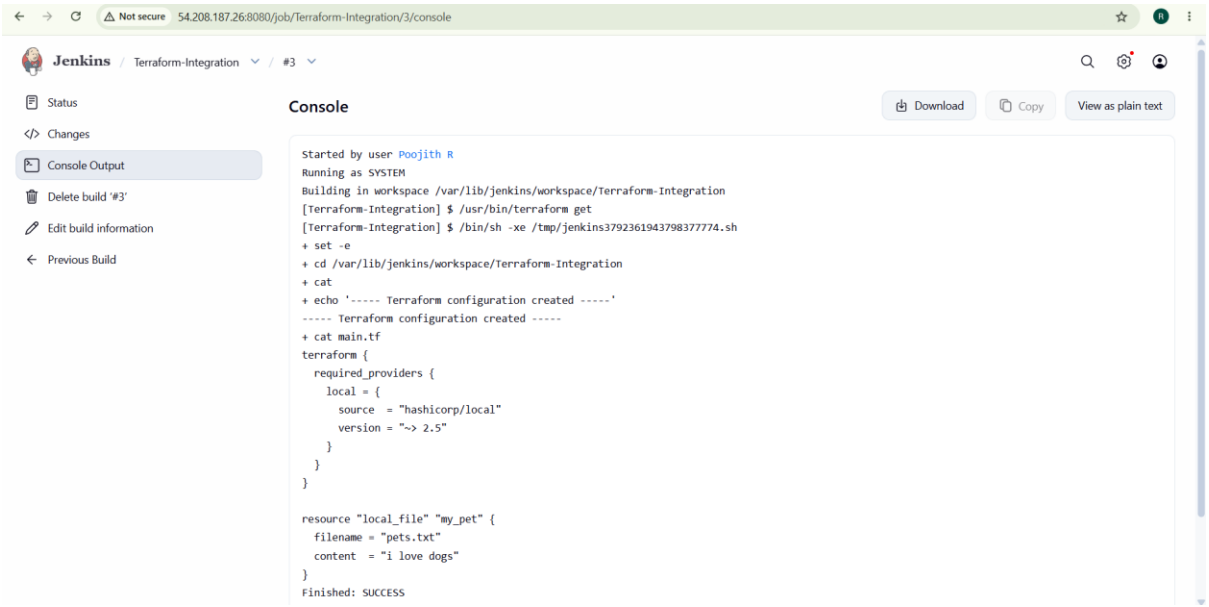
Final Verification

The Jenkins console output showed the Terraform plugin executing /usr/bin/terraform get, displaying the generated main.tf configuration, and finishing with:

Finished: SUCCESS







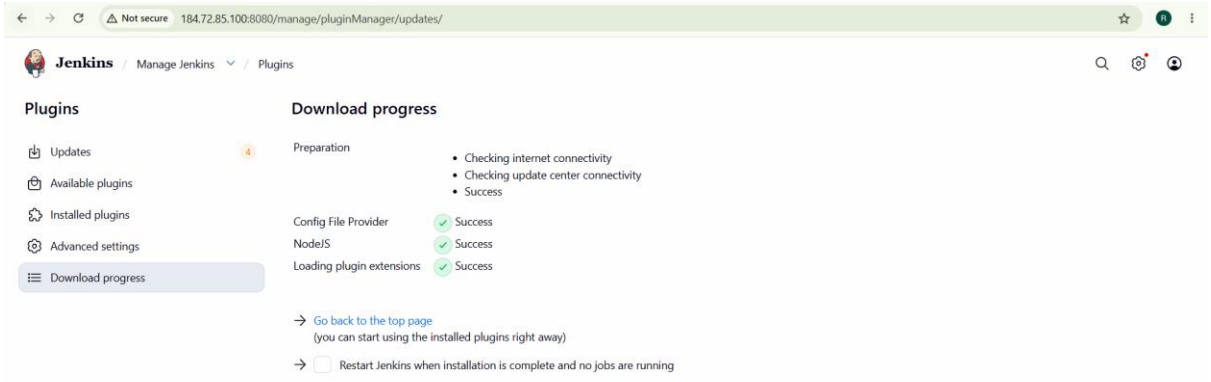
4. **Create a CI/CD pipeline for a Nodejs Application:**
   github.com/betawins/Trading-UI.git

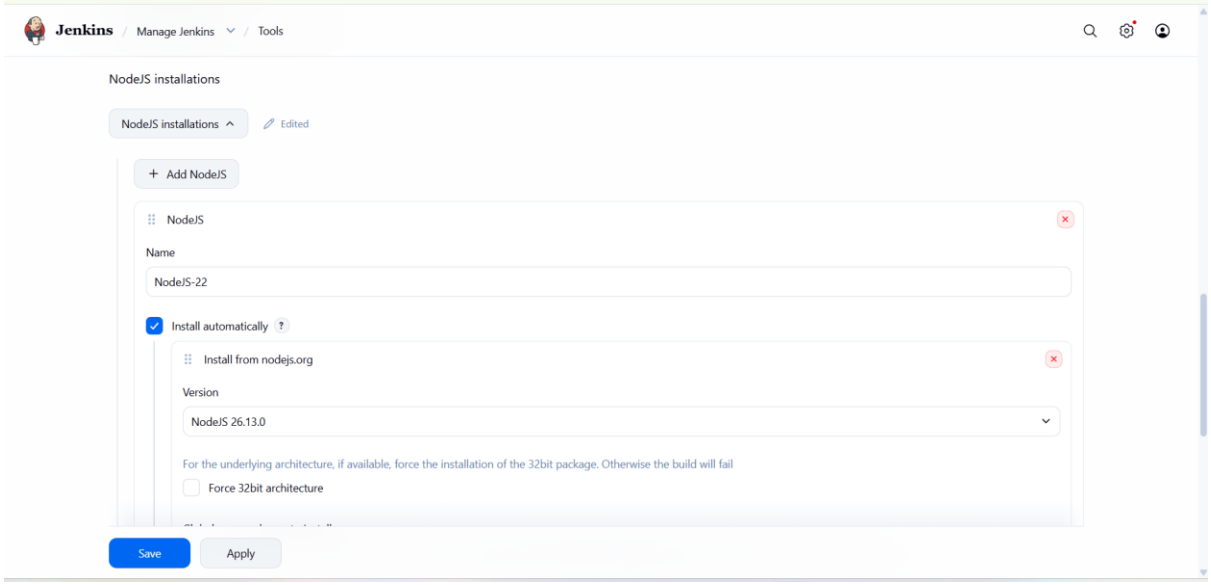
Implementation Steps:

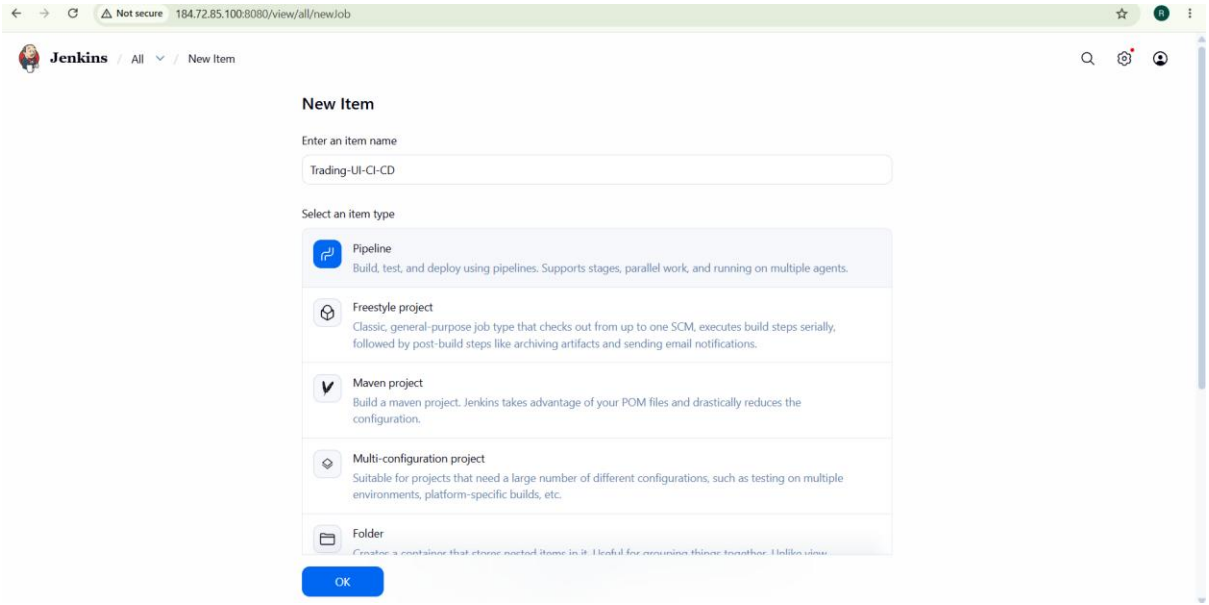
1. Opened the Jenkins dashboard and created a Pipeline job named Trading-UI-CI-CD.
2. Configured the NodeJS tool in Jenkins and selected NodeJS-22 in the Pipeline configuration.
3. Configured the Pipeline script to retrieve the application source code from the GitHub repository's master branch.
4. Added the Checkout stage to download the source code into the Jenkins workspace.
5. Added the Check Node and npm stage to verify the installed versions of Node.js and npm.
6. Added the Install Dependencies stage to install the application dependencies using npm install.
7. Added the Build stage to generate the production build using npm run build. Configured the required environment variables to support the existing React application.
8. Added the Test stage to execute the application's tests using npm test -- --watchAll=false.

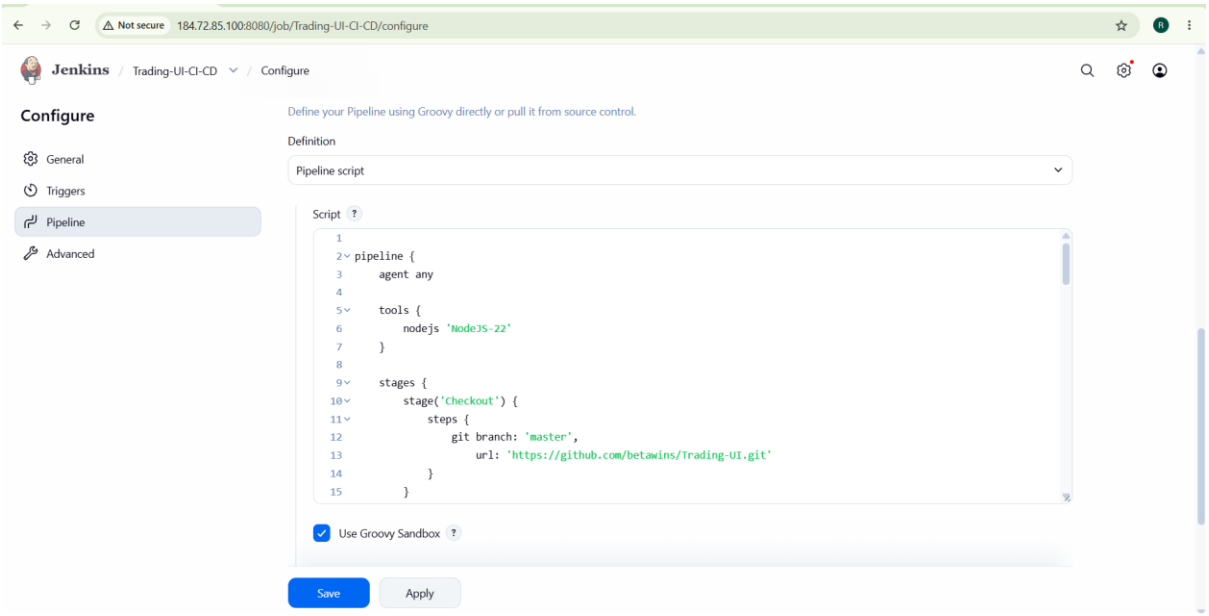
9. Configured error handling so that test failures would be reported in the console without stopping the entire Pipeline.
10. Added post-build messages to report the Pipeline status and completion.
11. Executed the Pipeline and verified the Checkout, dependency installation, Build, and Test stages in the Console Output.
12. Confirmed that the production build was generated successfully and the overall Jenkins build finished with the status SUCCESS.

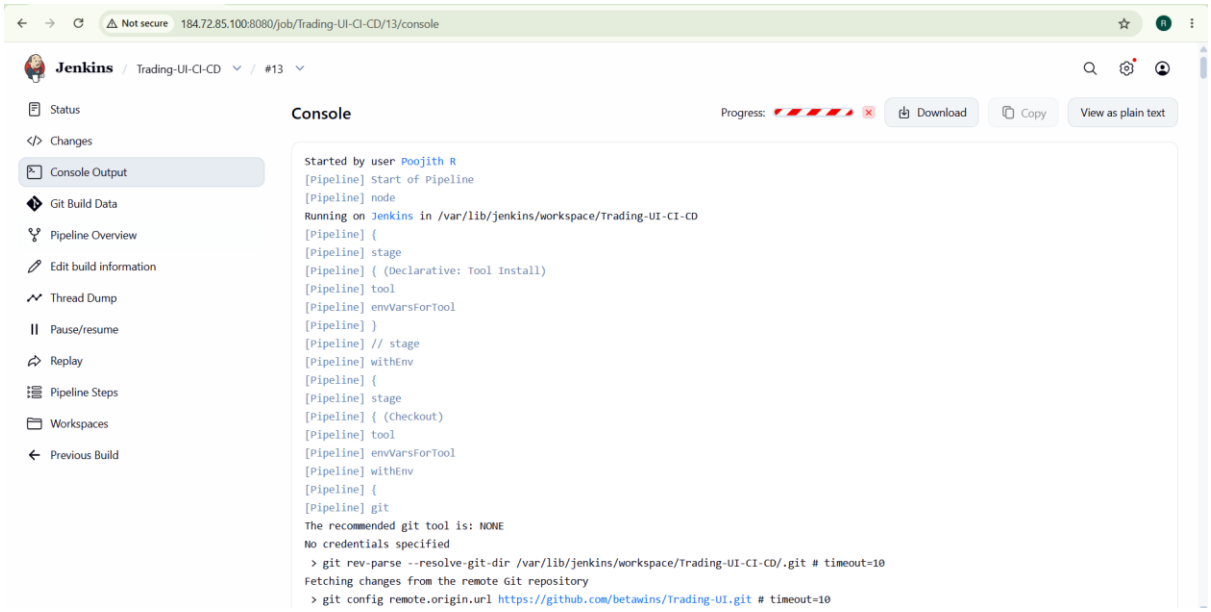
Observation: The test execution reported 12 passing tests and 1 failing test. The failure was recorded, but the Pipeline continued because of the configured error handling.

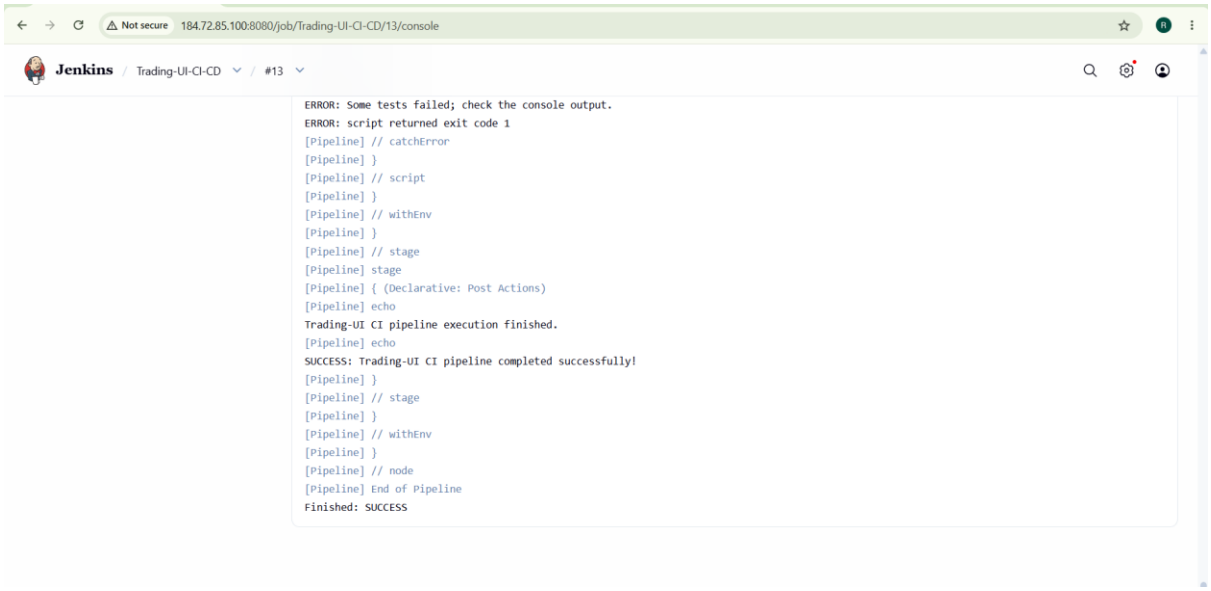





Not secure 184.72.85.100:8080/view/all/newJob
Jenkins / All / New Item
New Item
Enter an item name
Trading-UI-CI-CD
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
OK


Not secure 184.72.85.100:8080/job/Trading-UI-CI-CD/configure
Jenkins / Trading-UI-CI-CD / Configure
Configure
General
Triggers
Pipeline
Advanced
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
pipeline {
agent any
tools {
nodejs 'NodeJS-22'
}
stages {
stage('Checkout') {
steps {
git branch: 'master',
url: 'https://github.com/betawins/Trading-UI.git'
}
}
Use Groovy Sandbox
Save
Apply





5. Explain 10 Maven commands.

Command	Purpose
mvn -version	Display Maven and Java versions
mvn validate	Validate the project configuration
mvn compile	Compile the source code
mvn test	Run unit tests
mvn package	Package the application
mvn clean	Remove generated build files

mvn install	Install the artifact in the local Maven repository
mvn verify	Run verification checks
mvn deploy	Publish the artifact to a configured repository
mvn dependency:tree	Display project dependencies

Maven Commands

Maven is a build automation tool used mainly for Java projects. It helps compile source code, run tests, package applications, manage dependencies, and deploy build artifacts.

1. mvn -version

Purpose: Checks the installed Maven version and Java details.

mvn -version

Expected output (example):

Apache Maven 3.9.x

Maven home: /opt/maven

Java version: 21

2. mvn validate

Purpose: Checks whether the Maven project configuration is correct and the project is ready for the build process.

mvn validate

Expected result: BUILD SUCCESS if validation completes successfully.

3. mvn compile

Purpose: Compiles the project's main Java source code.

mvn compile

Expected result: Compiled class files are generally stored in the target/classes directory.

4. mvn test

Purpose: Runs the project's unit tests.

mvn test

Expected result: Maven displays the test results and reports BUILD SUCCESS if the tests pass.

5. mvn package

Purpose: Compiles the code, runs tests, and packages the application into a distributable artifact.

mvn package

Expected result: A JAR or WAR file is typically generated inside the target directory, depending on the project configuration.

6. mvn clean

Purpose: Removes files generated by the previous build, usually the target directory.

mvn clean

Expected result: The previous build output is removed.

7. mvn install

Purpose: Builds and packages the application, runs the required earlier lifecycle phases, and installs the artifact into the local Maven repository.

mvn install

Expected result: The artifact is stored in the local repository, usually ~/.m2/repository.

8. mvn verify

Purpose: Runs the build lifecycle through the verification phase, including applicable checks configured in the project.

mvn verify

Expected result: BUILD SUCCESS if the configured verification steps pass.

9. mvn deploy

Purpose: Builds and verifies the application, then publishes the artifact to a configured remote repository.

mvn deploy

Expected result: The artifact is uploaded to the configured repository, provided the repository and credentials are set up correctly.

Note: This is different from deploying an application directly to a running server.

10. mvn dependency:tree

Purpose: Displays the project's dependencies in a tree structure, including transitive dependencies.

mvn dependency:tree

Expected output (example):

com.example:my-app:jar:1.0

+- org.springframework:spring-core:jar:...

\- junit:junit:jar:...

This helps identify which libraries the project uses and where they come from.